

Dokumentacja techniczna aplikacji

System nadzoru pracy agentów ubezpieczeniowych

Zakres dokumentu: wymagania funkcjonalne i нефункционалне, role użytkowników, diagram bazy danych, opis tabel, API backendu oraz audyt zmian.

Technologie: Angular, TypeScript, Node.js, Express.js, SQLite

Baza danych: SQLite

Architektura: klient-serwer, REST API

Wersja dokumentu: 1.0

Spis treści

1. 1. Cel i zakres systemu
2. 2. Architektura i technologie
3. 3. Role użytkowników
4. 4. Wymagania funkcjonalne
5. 5. Wymagania niefunkcjonalne
6. 6. Przypadki użycia
7. 7. Diagram bazy danych
8. 8. Opis tabel bazy danych
9. 9. API backendu
10. 10. Audyt zmian
11. 11. Struktura projektu
12. 12. Propozycje dalszego rozwoju

1. Cel i zakres systemu

Celem aplikacji jest wspomaganie nadzoru nad pracą agentów ubezpieczeniowych. System umożliwia administratorowi zarządzanie agentami, tworzenie i planowanie spotkań, kontrolowanie ewidencji czasu pracy, prowadzenie dokumentacji spotkań oraz przeglądanie audytu zmian. Agent korzysta z osobnego panelu, w którym widzi tylko swoje spotkania, własny kalendarz, dokumentację i wpisy czasu pracy.

Zakres aplikacji został dopasowany do wymagań projektu zaliczeniowego, analogicznie do przykładowego systemu fitness, ale zastosowany do domeny agentów ubezpieczeniowych. System obejmuje encje biznesowe, przypisywanie spotkań agentom, harmonogram, dokumentację spotkań, role użytkowników, audyt zmian oraz dokumentację techniczną.

2. Architektura i technologie

- Frontend: Angular, TypeScript, HTML, CSS, Angular Router, FormsModule/ngModel, HttpClient.
- Backend: Node.js, Express.js, JavaScript, REST API.
- Baza danych: SQLite.
- Bezpieczeństwo haseł: bcryptjs, hashowanie haseł użytkowników.
- Obsługa plików: multer, folder backend/uploads, express.static.
- Kontrola wersji: Git, GitHub, plik .gitignore.

Aplikacja działa lokalnie w dwóch procesach: frontend Angular na porcie 4200 oraz backend Express na porcie 4000. Frontend komunikuje się z backendem przez REST API, np. `http://localhost:4000/api/agents`.

3. Role użytkowników

3.1. Administrator

- zarządzanie agentami: dodawanie, edycja, usuwanie, zmiana statusu;
- tworzenie, edycja i usuwanie spotkań;
- przypisywanie spotkań do aktywnych agentów;
- podgląd kalendarza wszystkich spotkań;
- zarządzanie notatkami i załącznikami spotkań;
- zarządzanie ewidencją czasu pracy agentów;
- przeglądanie audytu zmian.

3.2. Agent

- podgląd własnych spotkań i szczegółów spotkań;
- zmiana statusu własnego spotkania;
- podgląd własnego kalendarza;
- dodawanie, edycja i usuwanie notatek do własnych spotkań;
- dodawanie i usuwanie załączników do własnych spotkań;
- prowadzenie własnej ewidencji czasu pracy;
- zmiana hasła po pierwszym logowaniu.

4. Wymagania funkcjonalne

ID	Wymaganie
WF-01	System umożliwia logowanie użytkownika za pomocą adresu e-mail i hasła.
WF-02	System rozróżnia role administratora i agenta oraz przekierowuje użytkownika do właściwego panelu.
WF-03	System chroni trasy aplikacji na podstawie roli użytkownika.
WF-04	System wymusza zmianę hasła po pierwszym logowaniu agenta.
WF-05	System wymaga silnego hasła: minimum 8 znaków, mała litera, wielka litera, cyfra i znak specjalny.
WF-06	Administrator może dodawać, edytować, usuwać i przeglądać agentów.
WF-07	Administrator może ustawić status agenta jako aktywny lub nieaktywny.
WF-08	System umożliwia wyszukiwanie agentów po imieniu, nazwisku, e-mailu, telefonie i statusie.
WF-09	Lista agentów jest sortowana tak, aby aktywni agenci byli wyświetlani przed nieaktywnymi.
WF-10	Administrator może tworzyć spotkania i przypisywać je do aktywnych agentów.
WF-11	Administrator może edytować dane spotkania.
WF-12	Administrator może usuwać spotkania razem z powiązаныmi notatkami i załącznikami.
WF-13	Administrator i agent mogą zmieniać status spotkania po zatwierdzeniu przyciskiem Zapisz.
WF-14	Administrator ma dostęp do tygodniowego kalendarza wszystkich spotkań.
WF-15	Agent ma dostęp do tygodniowego kalendarza własnych spotkań.
WF-16	System umożliwia podgląd szczegółów spotkania.
WF-17	Administrator i agent mogą dodawać, edytować i usuwać notatki do spotkań.
WF-18	Administrator i agent mogą dodawać oraz usuwać załączniki do spotkań.

ID	Wymaganie
WF-19	Agent ma dostęp do modułu dokumentacji spotkań.
WF-20	Administrator może zarządzać ewidencją czasu pracy wszystkich agentów.
WF-21	Agent może zarządzać wyłącznie własnymi wpisami czasu pracy.
WF-22	System wylicza czas pracy na podstawie godziny rozpoczęcia i zakończenia.
WF-23	System waliduje, że godzina zakończenia pracy jest późniejsza niż godzina rozpoczęcia.
WF-24	System rejestruje najważniejsze operacje w audycie zmian.
WF-25	Administrator może przeglądać audyt zmian.
WF-26	System umożliwia wyszukiwanie wpisów audytu.
WF-27	System umożliwia wylogowanie użytkownika.

5. Wymagania niefunkcjonalne

ID	Wymaganie
WN-01	System działa w architekturze klient-serwer.
WN-02	Frontend i backend są rozdzielone na osobne części aplikacji.
WN-03	Komunikacja odbywa się przez REST API.
WN-04	Dane są przechowywane w relacyjnej bazie SQLite.
WN-05	Hasła są przechowywane w postaci hasha z użyciem bcryptjs.
WN-06	System waliduje dane po stronie frontendu i backendu.
WN-07	Interfejs jest spójny, czytelny i posiada profesjonalne nazewnictwo.
WN-08	Widoki list i audytu posiadają wyszukiwarkę oraz scrolowany obszar tabeli.
WN-09	Załączniki są przechowywane w folderze backend/uploads, a metadane w bazie danych.
WN-10	Repozytorium nie przechowuje node_modules, buildów, lokalnej bazy i uploadów użytkowników.

ID	Wymaganie
WN-11	System zapisuje audyt zmian obejmujący użytkownika, rolę, akcję, typ obiektu, identyfikator, opis i datę.
WN-12	Kod jest podzielony na moduły: auth, agents, meetings, notes, attachments, work-time, audit-logs.
WN-13	System wyświetla komunikaty błędów dla nieudanych operacji.
WN-14	Struktura aplikacji umożliwia dalszą rozbudowę, np. o raport PDF, JWT lub eksport danych.

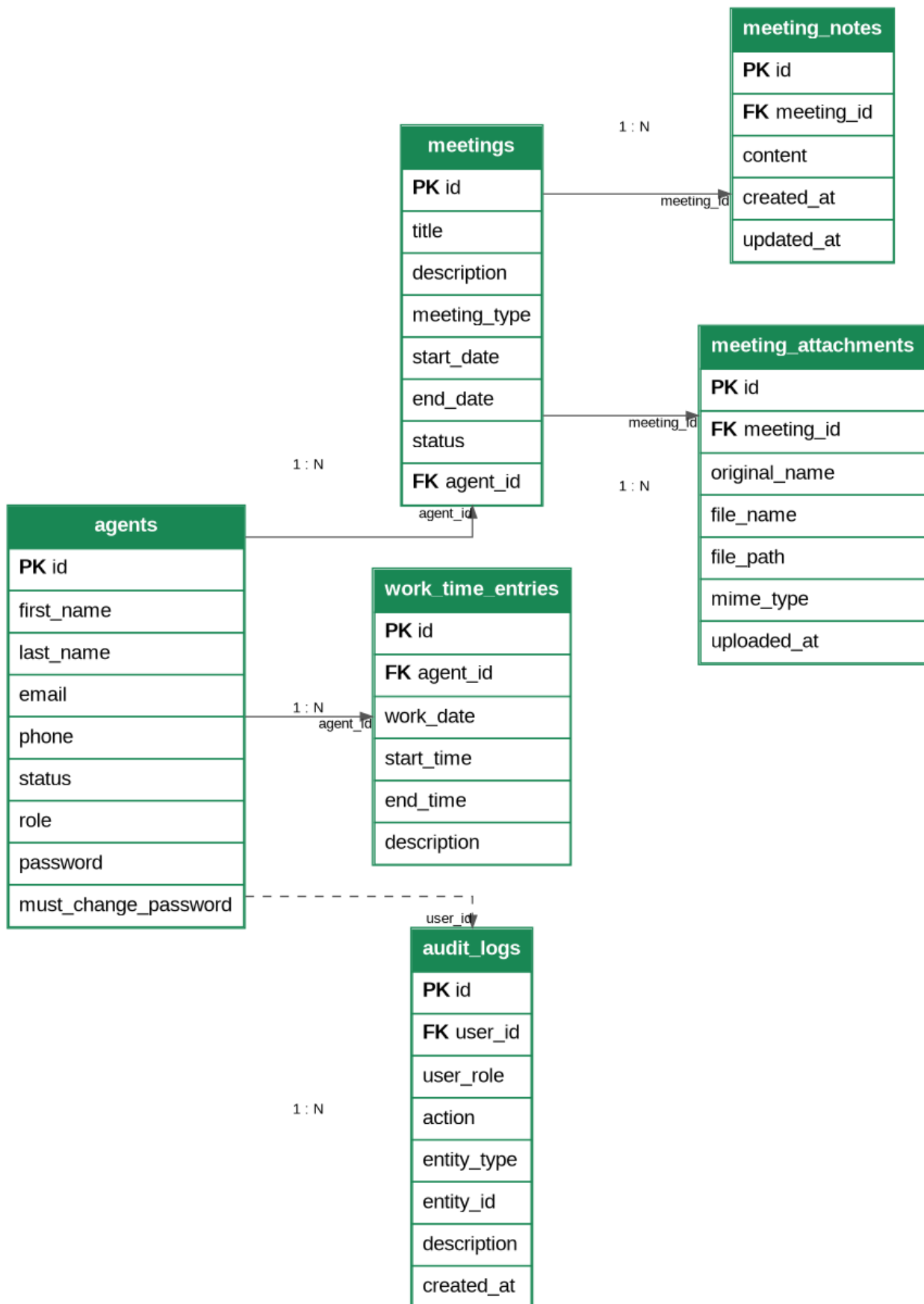
6. Przypadki użycia

ID	Nazwa	Aktor	Opis
PU-01	Logowanie użytkownika	Administrator, Agent	Użytkownik wpisuje e-mail i hasło, system weryfikuje dane i przekierowuje do właściwego panelu.
PU-02	Zmiana hasła po pierwszym logowaniu	Agent	Agent ustawia własne hasło spełniające wymagania bezpieczeństwa.
PU-03	Dodanie agenta	Administrator	Administrator tworzy konto agenta, podaje dane i hasło startowe.
PU-04	Edycja agenta	Administrator	Administrator modyfikuje dane agenta lub nadaje nowe hasło startowe.
PU-05	Dodanie spotkania	Administrator	Administrator tworzy spotkanie i przypisuje je do aktywnego agenta.
PU-06	Zmiana statusu spotkania	Administrator, Agent	Użytkownik wybiera nowy status spotkania i zatwierdza zmianę.
PU-07	Dodanie notatki	Administrator, Agent	Użytkownik dodaje notatkę do spotkania.
PU-08	Dodanie załącznika	Administrator, Agent	Użytkownik dodaje plik PDF lub obraz do

ID	Nazwa	Aktor	Opis
			spotkania.
PU-09	Dodanie wpisu czasu pracy	Administrator, Agent	Użytkownik dodaje wpis z datą, godziną rozpoczęcia i zakończenia.
PU-10	Przegląd audytu zmian	Administrator	Administrator przegląda historię operacji i korzysta z wyszukiwarki.

7. Diagram bazy danych

Poniższy diagram przedstawia główne tabele bazy danych oraz relacje między nimi. Relacje mają charakter jeden-do-wielu: jeden agent może mieć wiele spotkań i wpisów czasu pracy, jedno spotkanie może mieć wiele notatek i załączników, a użytkownik może wygenerować wiele wpisów audytu.



8. Opis tabel bazy danych

8.1. Tabela agents

Kolumna	Typ	Opis
id	INTEGER	Klucz główny.
first_name	TEXT	Imię użytkownika.
last_name	TEXT	Nazwisko użytkownika.
email	TEXT	Adres e-mail używany do logowania.
phone	TEXT	Numer telefonu agenta.
status	TEXT	Status: Aktywny lub Nieaktywny.
role	TEXT	Rola: ADMIN lub AGENT.
password	TEXT	Hash hasła.
must_change_password	INTEGER	Flaga wymuszenia zmiany hasła.

8.2. Tabela meetings

Kolumna	Typ	Opis
id	INTEGER	Klucz główny.
title	TEXT	Tytuł spotkania.
description	TEXT	Opis spotkania.
meeting_type	TEXT	Typ spotkania.
start_date	TEXT	Data i godzina rozpoczęcia.
end_date	TEXT	Data i godzina zakończenia.
status	TEXT	Status spotkania.
agent_id	INTEGER	Klucz obcy do agents.id.

8.3. Tabela meeting_notes

Kolumna	Typ	Opis
id	INTEGER	Klucz główny.
meeting_id	INTEGER	Klucz obcy do meetings.id.
content	TEXT	Treść notatki.
created_at	TEXT	Data utworzenia.

Kolumna	Typ	Opis
updated_at	TEXT	Data ostatniej aktualizacji.

8.4. Tabela meeting_attachments

Kolumna	Typ	Opis
id	INTEGER	Klucz główny.
meeting_id	INTEGER	Klucz obcy do meetings.id.
original_name	TEXT	Oryginalna nazwa pliku.
file_name	TEXT	Nazwa pliku zapisanego na serwerze.
file_path	TEXT	Ścieżka do pliku.
mime_type	TEXT	Typ MIME pliku.
uploaded_at	TEXT	Data dodania załącznika.

8.5. Tabela work_time_entries

Kolumna	Typ	Opis
id	INTEGER	Klucz główny.
agent_id	INTEGER	Klucz obcy do agents.id.
work_date	TEXT	Data pracy.
start_time	TEXT	Godzina rozpoczęcia.
end_time	TEXT	Godzina zakończenia.
description	TEXT	Opis wpisu czasu pracy.

8.6. Tabela audit_logs

Kolumna	Typ	Opis
id	INTEGER	Klucz główny.
user_id	INTEGER	Klucz obcy do agents.id, użytkownik wykonujący operację.
user_role	TEXT	Rola użytkownika w momencie operacji.
action	TEXT	Typ operacji.
entity_type	TEXT	Typ obiektu.
entity_id	INTEGER	Identyfikator obiektu.
description	TEXT	Opis operacji.
created_at	TEXT	Data wykonania operacji.

9. API backendu

Backend udostępnia REST API dostępne pod adresem bazowym <http://localhost:4000/api>. Poniżej przedstawiono najważniejsze endpointy systemu.

9.1. Autoryzacja

Metoda	Endpoint	Opis
POST	/api/auth/login	Logowanie użytkownika.
POST/PATCH	/api/auth/change-password	Zmiana hasła po pierwszym logowaniu.

9.2. Agenci

Metoda	Endpoint	Opis
GET	/api/agents	Pobranie listy agentów.
GET	/api/agents/:id	Pobranie pojedynczego agenta.
POST	/api/agents	Dodanie agenta.
PUT	/api/agents/:id	Edycja agenta.
DELETE	/api/agents/:id	Usunięcie agenta.

9.3. Spotkania

Metoda	Endpoint	Opis
GET	/api/meetings	Pobranie wszystkich spotkań.
GET	/api/meetings/agent/:agentId	Pobranie spotkań konkretnego agenta.
GET	/api/meetings/:id	Pobranie szczegółów spotkania.
POST	/api/meetings	Dodanie spotkania.
PUT	/api/meetings/:id	Edycja spotkania.
PATCH	/api/meetings/:id/status	Zmiana statusu spotkania.
DELETE	/api/meetings/:id	Usunięcie spotkania z dokumentacją.

9.4. Notatki

Metoda	Endpoint	Opis
GET	/api/meeting-notes/meeting/:meetingId	Pobranie notatek spotkania.
POST	/api/meeting-notes	Dodanie notatki.
PUT	/api/meeting-notes/:id	Edycja notatki.
DELETE	/api/meeting-notes/:id	Usunięcie notatki.

Metoda	Endpoint	Opis

9.5. Załączniki

Metoda	Endpoint	Opis
GET	/api/meeting-attachments/meeting/:meetingId	Pobranie załączników spotkania.
POST	/api/meeting-attachments	Dodanie załącznika.
DELETE	/api/meeting-attachments/:id	Usunięcie załącznika.

9.6. Czas pracy

Metoda	Endpoint	Opis
GET	/api/work-time	Pobranie wszystkich wpisów czasu pracy.
GET	/api/work-time/agent/:agentId	Pobranie wpisów czasu pracy agenta.
POST	/api/work-time	Dodanie wpisu czasu pracy.
PUT	/api/work-time/:id	Edycja wpisu czasu pracy.
DELETE	/api/work-time/:id	Usunięcie wpisu czasu pracy.

9.7. Audyt zmian

Metoda	Endpoint	Opis
GET	/api/audit-logs	Pobranie historii operacji systemowych.

10. Audyt zmian

Audyt zmian zapisuje najważniejsze operacje wykonywane w systemie. Każdy wpis zawiera identyfikator użytkownika, rolę, typ akcji, typ obiektu, identyfikator obiektu, opis oraz datę wykonania operacji. Administrator może przeglądać audyt w panelu administracyjnym oraz korzystać z wyszukiwarki.

10.1. Rejestrowane akcje

Akcja	Znaczenie
CREATE_AGENT, UPDATE_AGENT, UPDATE_AGENT_WITH_PASSWORD, DELETE_AGENT	Operacje na agentach.
CREATE_MEETING, UPDATE_MEETING, UPDATE_MEETING_STATUS, DELETE_MEETING	Operacje na spotkaniach.

Akcja	Znaczenie
CREATE_NOTE, UPDATE_NOTE, DELETE_NOTE	Operacje na notatkach.
UPLOAD_ATTACHMENT, DELETE_ATTACHMENT	Operacje na załącznikach.
CREATE_WORK_TIME, UPDATE_WORK_TIME, DELETE_WORK_TIME	Operacje na czasie pracy.

11. Struktura projektu

```

insurance-agents-app/
├── backend/
│   ├── src/
│   │   ├── index.js
│   │   ├── routes/
│   │   │   ├── auth.routes.js
│   │   │   ├── agents.routes.js
│   │   │   ├── meetings.routes.js
│   │   │   ├── meeting-notes.routes.js
│   │   │   ├── meeting-attachments.routes.js
│   │   │   ├── work-time.routes.js
│   │   │   └── audit-logs.routes.js
│   │   ├── utils/
│   │   └── audit.js
│   └── frontend/
│       ├── src/app/
│       │   ├── guards/
│       │   ├── pages/
│       │   │   ├── admin-dashboard/
│       │   │   ├── admin-agents/
│       │   │   ├── admin-meetings/
│       │   │   ├── admin-calendar/
│       │   │   ├── admin-work-time/
│       │   │   ├── admin-audit-logs/
│       │   │   ├── agent-dashboard/
│       │   │   ├── agent-calendar/
│       │   │   ├── agent-work-time/
│       │   │   └── agent-documents/
│       │   └── app.routes.ts

```

12. Propozycje dalszego rozwoju

- dodanie autoryzacji backendowej opartej o token JWT;
- dodanie eksportu raportów do PDF lub CSV;
- dodanie filtrowania spotkań po statusie, agencji i terminie;
- dodanie paginacji dla dużych list danych;
- wydzielenie wspólnych komponentów dla notatek, załączników i tabel;
- dodanie testów automatycznych dla backendu i frontendu;
- rozbudowa audytu o porównanie starych i nowych wartości pól.